L1 – C++ ,technical, problem , design pattern, data structure , problem solving, coding

L2 –Cluod, kubctl command, space , docker , pod , scalup ,scal down ,AWS Queue cutter.

I

If there are 50 cunstomer how do you do ymal file.

For performance what tool used?

Why we used factory DP we can directly create object

How to update each vm/container

IF u passing address to function then you needed shared memory ?

If you using UDP does it required any ethernet ?

how web page is loaded in your system

How this pointer is declared and where?

Back after a long time! — C++ Interview Hack

If you're preparing for a C++ interview, here's a simple trick:

Don't just prepare “What is virtual function?”

Prepare the WHY + HOW + WHAT IF.

For example:

 Basic:
“What is a virtual function?”

 Better preparation:
→ Why do we need it?
→ How does it work internally?
→ What is a vtable/vptr?
→ What happens if the destructor isn't virtual?
→ Can a constructor be virtual? Why not?
→ What's the performance cost?

 Interviewers often start with a simple question and then go deeper.

So whenever you study a C++ concept, prepare it in 3 layers:

 Level 1: Definition
 Level 2: Practical implementation
 Level 3: Internal working + edge cases

This approach works especially well for:
"virtual" functions • pointers • references • RAII • smart pointers • move semantics • STL • multithreading

Don't prepare answers. Prepare the concept.

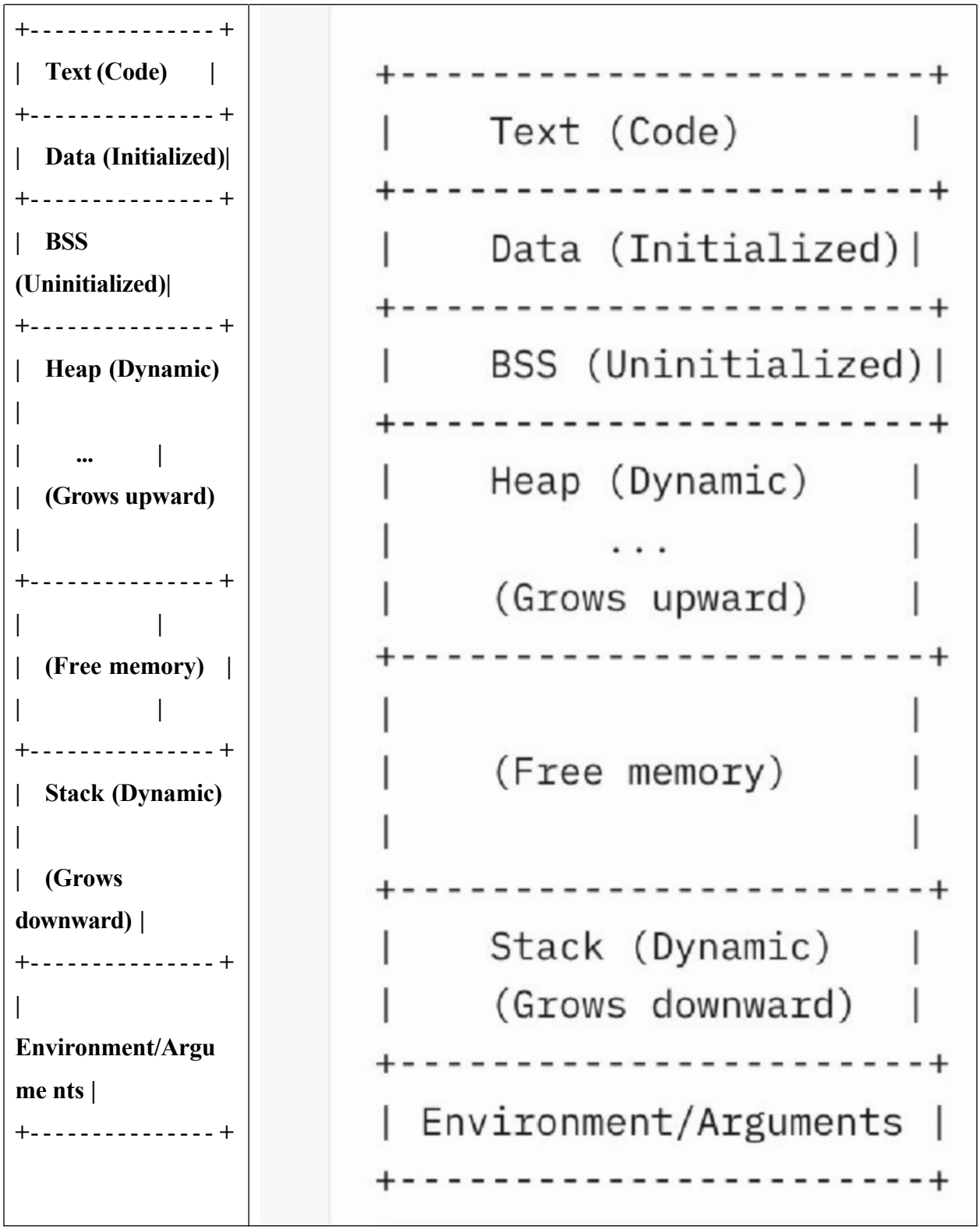
That's the real C++ interview hack. □

Example: Long to Wide Transformation

Input (`long_data`):

Student	Subject	Score
Alice	Math	90
Alice	Science	85
Bob	Math	75
Bob	Science	80

SAS Code:



What is difference between pointer & reference c++ ?

=>

Pointer	Refernce
It can be Null	It CAN NOT NULL
It can be de-refernce using * operator	It doesn't required to de-refernce
Arithmatic operation like ++, - – possible	No Arithematic operaton not possible

Pojnter to Pointer are possible	**Refernce to refernce not possible .**
Pointer occupy its own address	**Refernce don't have its own address.**
It can be re assign to other address,	**It can't re –refernce to other variable.**

What std::forward in c++ ?

=> it is simpler to function overloading like **depending parameter passing** , it decided which function to called . It work only in template , Here wrapper function is decide whether to call *void process(int& x)* or *void process(int&& x)* . It depended on parameter passing wrapper(a); // Prints: Lvalue (named variable)
wrapper(10); // Prints: Rvalue (temporary value)

```
#include <iostream>
#include <utility>

void process(int& x) {
    std::cout << "Lvalue (named variable)\n";
}

void process(int&& x) {
    std::cout << "Rvalue (temporary value)\n";
}

template <typename T>
void wrapper(T&& arg) {
    // std::forward preserves whether 'arg' was originally an lvalue or rvalue
    process(std::forward<T>(arg));
}

int main() {
    int a = 5;

    wrapper(a);   // Prints: Lvalue (named variable)
    wrapper(10);  // Prints: Rvalue (temporary value)
 //int &&X =20    , as this statement is valid
}
```

Q. We can used move for Rvalue from above code ,then why forward required? :

Q. Why reserve is used . code ,then why forward required? :
=>

```
  // std::vector<thread> threads(5)  // valide statment but
// but here above line size , capacity is 5 , when emplay_back / push_back new memory allocated after 5 object means 6,7,9...  , So after for loop ,you see you have size ,capacity is 10.
Also when you have separate join then your code is crash .
    std::vector<thread> threads;
    threads.reserve(5);

    cout<<" size :"<<threads.size()<<" capacity :"<<threads.capacity()<<"\n";
```

```
   for(int i=0; i< 5 ; ++i){

     //  threads.emplace_back( (print)).join();  //this is also correct  but for
           threads.emplace_back( (print).  // not join
       //threads.push_back(std::thread(print));
   }
 cout<<" size :"<<threads.size()<<" capacity :"<<threads.capacity()<<"\n";
```

```
#include <iostream>
#include <string>
#include <memory>
using namespace std;
      class Player{
            public:
       Player(string name, int lid ):m_name(name),m_licenced(lid){}

        void virtual display()=0;
       virtual  ~Player()=0;//{ // you can't declare defination at same time.
                          // pure virtual defination is outside of class.
            // cout<< "\n ~Base \n";
       // }

        protected:
        string m_name;
        int m_licenced;
      };

      //outside definition destructor
      Player::~Player()
         { cout<< "\n ~Base
         \n";
      }

      class Stricker : public
         Player{ int m_goal;
         public:
         Stricker(string name,int lid, int goal):m_goal(goal),Player(name,lid){

           // ::Player(name,lid);
         }
         void display() {
            cout<< "\n override \n";
         }
~Stricker(){
           cout<< "\n ~Derive \n";

         }

      };
```

```
int main() {
    // Write C++ code here
    std::cout << "Try programiz.pro";

    string name ="Sagar";

    int licenced =1;
    int goal =2;
    unique_ptr<Stricker> obj= make_unique<Stricker> (name,licenced,goal);
   //Player *obj = new Stricker(name,licenced,goal);

    obj->display();

   // delete obj;

    return 0;

}
```

Q. from vector remove at position :

=>std::vector<int> vec = {1, 2, 3, 4};
auto it = vec.begin() + 1; // Iterator to 2
vec.erase(it); // vec now {1, 3, 4}

Q. How do you optimize a C++ function for speed?

=> Inline small functions, use const references, leverage std::move, minimize dynamic allocation, and profile with tools like gprof. Discuss compiler optimizations (e.g., -O2).

Q. What is RAII?

=> "Resource Acquisition Is Initialization" — tying resource lifetime (memory, file handles, locks) to object lifetime, so the destructor automatically releases the resource, even if an **exception is thrown** .

Q. What's the difference between shallow copy and deep copy? Why does it matter?

=> Shallow copy **duplicates the pointe**r, not the pointed-to data — both objects end up pointing to the same memory, causing double-free bugs. Deep copy allocates new memory and copies the actual data. Matters whenever a class manages a raw pointer/resource — you need a custom copy constructor and copy-assignment operator (or disable them).

Q. vector vs. list vs. deque — when do you use each?

- vector: contiguous memory, fast random access, slow insert/delete in the middle.
- list: doubly linked list, fast insert/delete anywhere, no random access.
- deque: double-ended, fast insert/delete at both ends, still supports random access.

Q. What's an iterator invalidation, and give an example.

=> When the container changes its internal memory layout (resizes, inserts, deletes elements), but iterator already point to begin() then it++ is invalided.
Ex.

```
for (auto it = v.begin(); it != v.end(); ++it) {
```

```
if (*it == 3) {
v.erase(it); // BUG: erase invalidates 'it'
 } }
```

Solution is

```
It = v.erase(it);
```

Q How do std::lock_guard and std::unique_lock differ?

=> *lock_guard* can automatically unlock we goes out of scope. *Unique_lock* also same unlock. But *Unique_lock,* unlock() explicitly is if you want to release the lock before the end of scope . .Also used when we to have used *condition_variable(cv)*

Quick comparison

	`lock_guard`	`unique_lock`
Locks on construction	Yes	Yes (unless `defer_lock`)
Unlock early manually	☐ No	☐ Yes
Re-lock later	☐ No	☐ Yes
Movable / transferable	☐ No	☐ Yes
Works with `condition_variable`	☐ No	☐ Yes
Overhead	Lower	Slightly higher (tracks lock state)

Rule of thumb

- Use **lock_guard** when you just need to lock a mutex for a scope and don't need any flexibility — this is the common case, and it's cheaper.
- **Use unique_lock** when you need to unlock early, lock conditionally/later, transfer ownership, or use it with condition_variable.

Q what is namespace , why it used ?

=>it is ways to group related names together like classes, functions,variable . It is used to prevent naming conflicts and organize code into logical modules,

Q. what is rvalue and lvalue explain with example

=> **Simple rule of thumb : If you can take its address with &, it's an lvalue. If you can't, it's an rvalue.**

```
int x = 10;
int z = 10;       // 10 is an rvalue — it's a temporary literal, no address, no nameint
w = x + 5;      // (x + 5) is an rvalue — the result is temporary, computed and discarded after assignment
```

cpp

```
&x;       // ☐ works, x is an lvalue
&(x + 5); // ☐ compile error — you can't take the address of a temporary
&10;      // ☐ compile error — same reason
```

Q. what is race condition?

=> when two or more process or thread access & modify same shared resource at same time, without proper coordinator leads to incorrect result

Q. what is initialize list ?

=> it is used initialize const, reference & base class member ,

Q. What is connection pool?

=> instead of opening and closing a new connection for every query.it set of pre-established database connections is maintained and reused across application requests

Q .observers.erase(std::remove(observers.begin(), observers.end(), observer), observers.end());

Why erase + remove used ?

=> **`std::remove`** does NOT actually remove anything from the container (misleading name!). It:

1. Shifts all elements that **don't** match `observer` to the front of the range
2. Returns an iterator pointing to the new "logical end" of the valid data
3. Leaves the *tail* of the vector in an unspecified/leftover state (size doesn't shrink)

`erase` then physically deletes that leftover tail, from the returned iterator to the real `end()`, shrinking the vector's size.

Visual Example

Say you have `observers = [A, B, C, B, D]` and you want to remove `B`:

Before: [A, B, C, B, D]

std::remove(B) rearranges to:

[A, C, D, ?, ?] ← returns iterator pointing here (index 3)

(valid data) (leftover garbage)

erase(iter, end()) chops off the "?, ?" part:

After: [A, C, D]

Q. Why below code print 1 to 4 in vector ?

=>

```
vector<int> v = {1,2,3,4};

std::vector<int>::iterator it3 = v.begin(); // points to first element (1)
std::vector<int>::iterator it4 = v.end();   // points *past* the last element

while (it3 < it4) {
    cout << " " << *it3;
    it3++;
}
```

Key points:
v.begin() → iterator to the first element (1).
v.end() → iterator **one after the last element.** It does not point to a valid element; it's just a sentinel that tells you "you're done".
Your loop condition: while (it3 < it4) means "keep going while it3 is before it4".
So the loop runs for:
it3 at 1 → prints 1, then it3++
it3 at 2 → prints 2, then it3++
it3 at 3 → prints 3, then it3++
it3 at 4 → prints 4, then it3++
Now it3 == it4 (both are end()), so it3 < it4 is false → loop stops.
That's exactly why you see: 1 2 3 4.

TODO : memory pool

Q . What is aggregate initialization ?
=>we can initialize class member variable **without** parameterized constructor.

```
        ex .class  A {
        String name ; };
int main () {
        A obj { "Sagar" };   }//This Syntax
```

Note : 1.Here we **NOT** required parameterized constructor. if define default constructor then above is not valid , it give error.
2. If we add a user-defined constructor, the class is no longer an aggregate (under C++14 rules), and then a matching constructor is required.
3. An aggregate cannot have user-provided constructors.

Q. Why #pragma used?
=> if header files included **multiples times**, across different file then compiler thrown error duplicate declaration error. #pragma tell compiler **only one copy at compilation** unit.

Q. Memory allow allocated on heap, NOT stack ?
=> for stack , memory need call destructor, so make destructor as private.

```
        #include <iostream>

class HeapOnly {
private:
   // Make destructor private to prevent stack allocation
   ~HeapOnly() {
      std::cout << "HeapOnly destroyed\n";
   }
```

```
public:
    HeapOnly() {
        std::cout << "HeapOnly created on heap\n";
    }

    // Public method to safely delete the object
    void destroy() {
        delete this;
    }
};

int main() {
    // ------------------------------------------------------------
    // 1. Stack allocation (Will NOT compile)
    // ------------------------------------------------------------
     HeapOnly obj; // Error: 'HeapOnly::~HeapOnly()' is private

    // ------------------------------------------------------------
    // 2. Heap allocation (Allowed)
    // ------------------------------------------------------------
    HeapOnly* ptr = new HeapOnly();

    // delete ptr; // Error: destructor is private

    // Proper cleanup via public member function
    ptr->destroy();

    return 0;
}
```

Q. Memory allow allocated on stack, NOT heap ? :

=> For NOT heap ,so make *operator new* private.

```
#include <iostream>

class StackOnly {
public:
    StackOnly() {
        std::cout << "StackOnly created\n";
    }

    ~StackOnly() {
        std::cout << "StackOnly destroyed\n";
    }

    // Disable standard dynamic allocation (heap) OR make private
    void* operator new(size_t) = delete;
    void* operator new[](size_t) = delete;

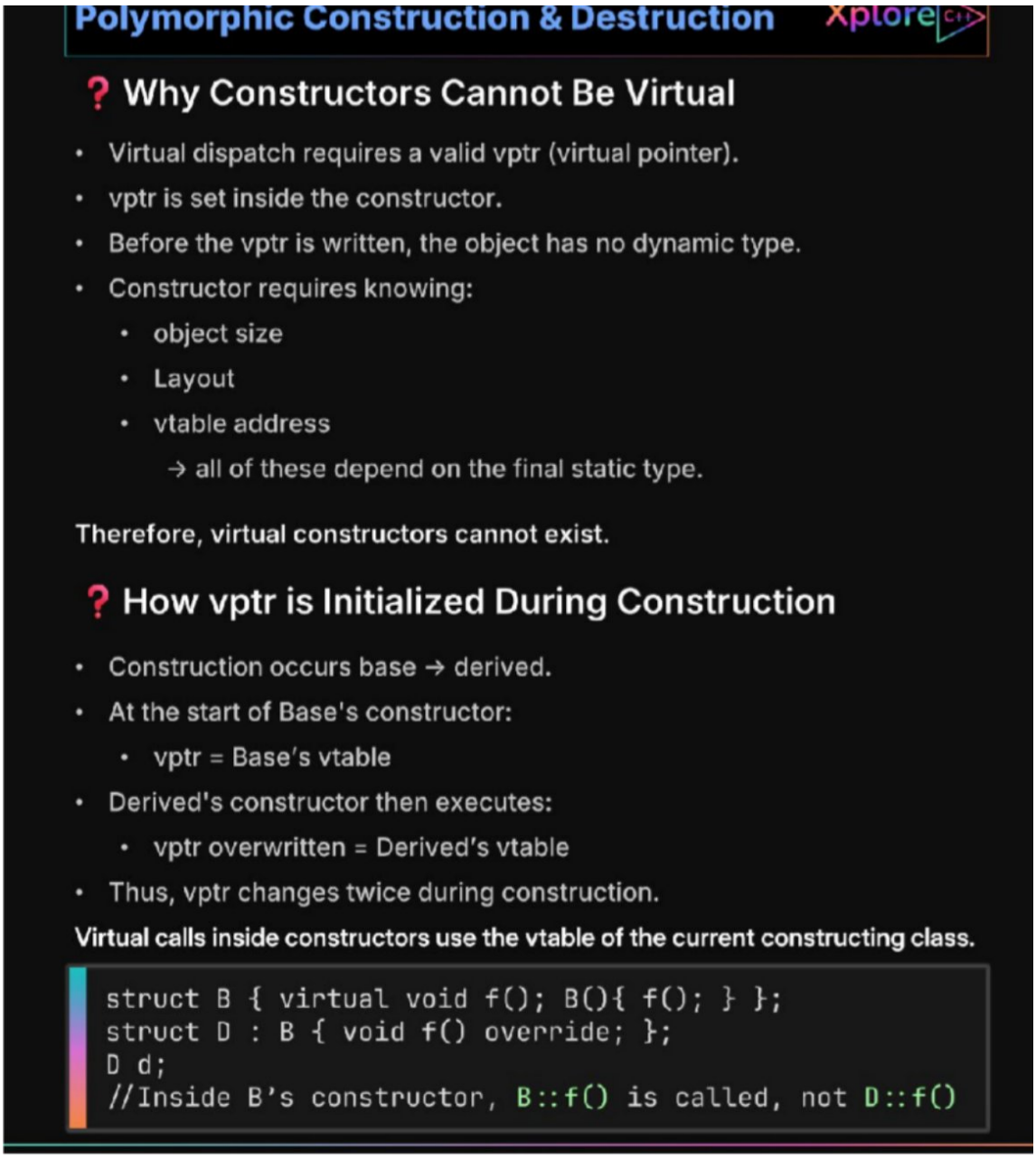
    // Optional: Also disable placement new to prevent allocating inside pre-allocated heap
memory
    void* operator new(size_t, void*) = delete;
    void* operator new[](size_t, void*) = delete;
};

int main() {
```

```
    // ------------------------------------------------------------
    // 1. Stack allocation (Allowed)
    // ------------------------------------------------------------
    StackOnly obj; // Works perfectly

    // ------------------------------------------------------------
    // 2. Heap allocation (Will NOT compile)
    // ------------------------------------------------------------
    // StackOnly* ptr = new StackOnly();     // Error: use of deleted function 'operator new'
    // StackOnly* arr = new StackOnly[5];    // Error: use of deleted function 'operator new[]'

    return 0;
}
```



Polymorphic Construction & Destruction

Xplore C++

❓ Why Destructors Should Be Virtual in Polymorphic Bases

If deleting via base pointer:

```
Base* b = new Derived;
delete b;       // MUST call Derived::~Derived()
```

Without virtual `~Base()`, only Base's destructor runs → **UB.**

Virtual destructor ensures:

1. `Derived::~Derived()`
2. `Base::~Base()`

❓ Construction and Destruction Order

- Construction: Base → Intermediate → Derived
- Destruction: Derived → Intermediate → Base
- This ensures:
  - Derived resources are released first
  - Base cleans up last
  - Consistent RAII behavior

Polymorphic Construction & Destruction

Xplore C++

❓ Pure Virtual Destructors

Allowed:

```
struct Base {
    virtual ~Base() = 0;
};
```

But must still have a definition:

```
Base::~Base() {}
```

Reason:

- When destroying a Derived object, `Base::~Base()` must still run.

❓ Virtual Inheritance Destructor Order

If A is a virtual base:

- A constructed once, before all non-virtual bases.
- A destroyed once, after all derived classes.

Destroy order:

```
MostDerived::~
→ Non-virtual bases
→ Virtual bases
```

Important for diamond inheritance.

Polymorphic Construction & Destruction

Xplore C++

❓ Why “delete this" Is Dangerous

Using delete this inside a member function is only safe if:

- object was allocated via new
- object is not part of a larger object
- the function is the last use of the object
- object is not allocated on stack or as global/static

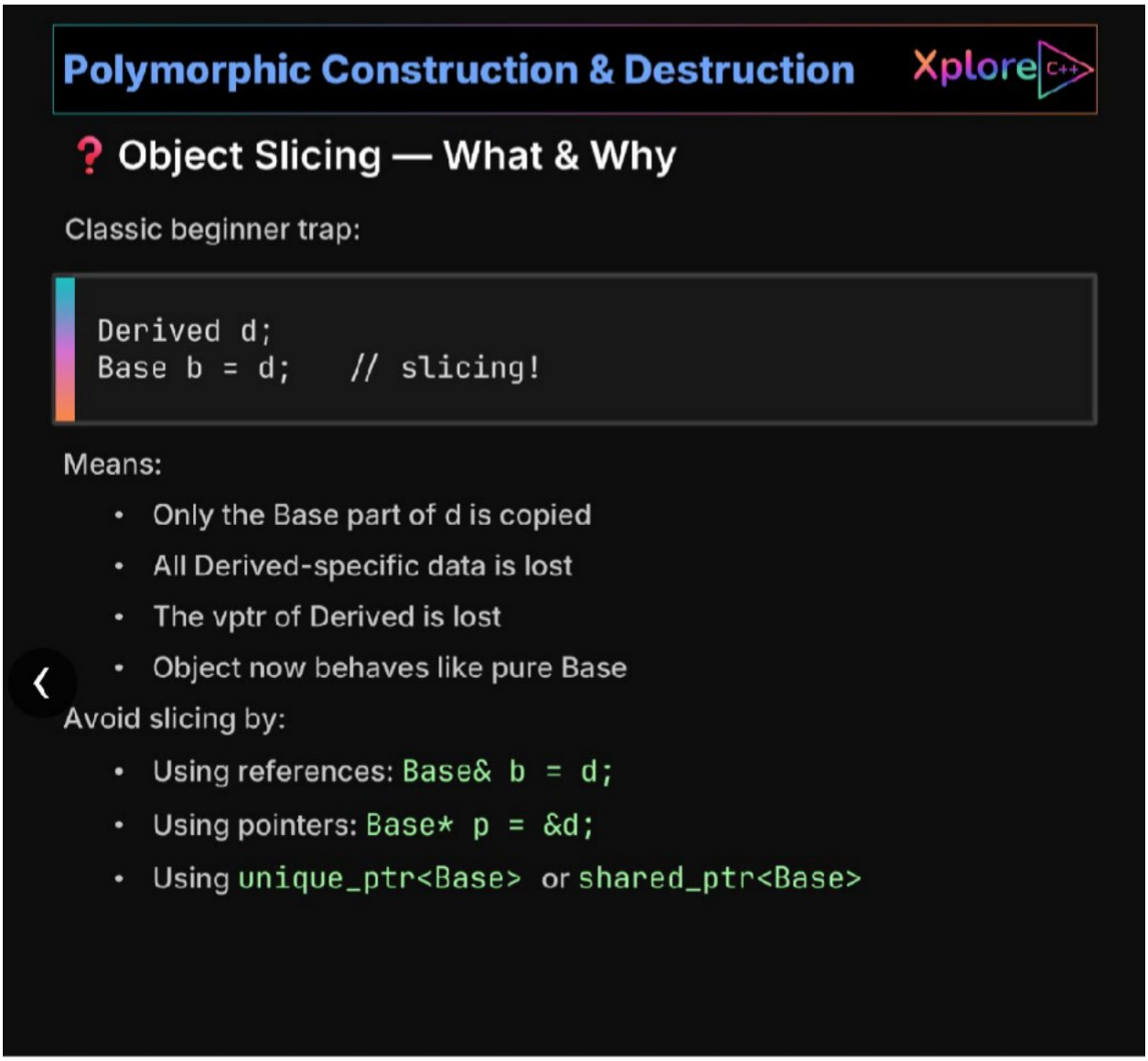
Else → **UB**, double-free, crashes.

❓ Object Lifetime & Polymorphism Rules

Key lifetime concepts:

- Object does not become its derived type until after base constructor finishes.
- Virtual dispatch is disabled inside constructors/destructors.
- Dynamic type is stable only:
  - after construction finishes
  - before destruction begins
- After destruction begins, vptr is rewritten to base's vtable.

This explains why you must avoid calling overridden methods inside destructors.



What is C++11 new Features ?

=> 1. Move
2. Lambda
3. Auto
4. Override
5. Range base loop
6. Smart pointer – unique_ptr, shared_ptr, weak_ptr
7. Thread std library

What is C++14 new Features ?

=>1. auto as parameter passing also auto at return
2. Make concept like make_unique ,make_shared, make_weak which provides **exception safety** and can be **more efficient** than using ***new*** directly.

What is thread shared some memory stack .

Q 1. concept of a virtual function

=>A virtual function in C++ is a member function in a base class that can be overridden in a derived class. It enables **runtime polymorphism**, allowing the correct function to be called **based on the object's actual type, not the pointer/reference type.**

Q2 .Why Interface is need as are creating object of concrete class, So we can directly call member function of that concrete class .

=> as we satisfied design principle open –close principle like open for enhancement close for modification ex. Database like SQL, Postgres, concendra so we easily enhance code , that why we used interface.

For_each	Syntax :
	for_each(begin, end ,function_name/lambda)
	Ex.
	For array
	for_each(array,array+size [](int x) { cout<<x});
	for Vector
	for_each(vec.begin(),v.end(),[](int x) { cout<<x});

Time Complexity :

- If your code **divides** the problem size **by 2**, the time complexity is O(log n) with an implied base of 2.
- If it divides by 3, 4, 5, or any constant, it's still O(log n) because the base only affects the **constant** factor, which Big O ignores.

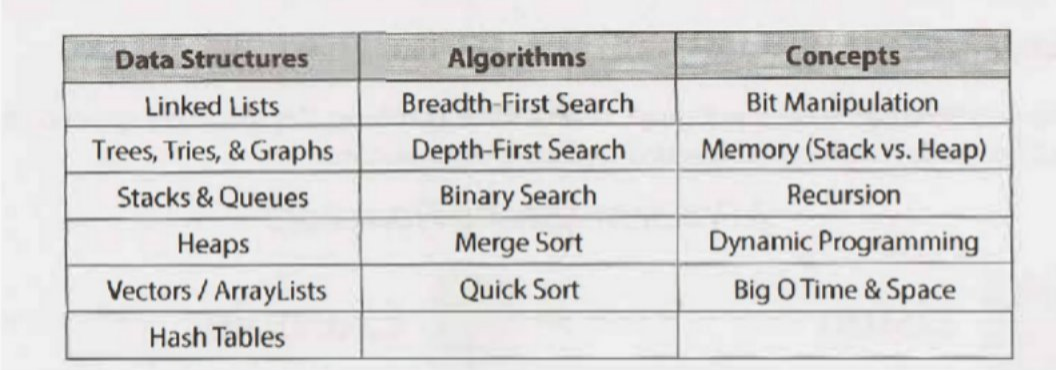
Common Time Complexities

- **O(1)**: Constant (e.g., accessing an array index).

- **O(log n)**: Logarithmic (e.g., binary search).
- **O(n)**: Linear (e.g., searching an unsorted list).
- **O(n log n)**: Linearithmic (e.g., efficient sorting like mergesort).
- **O(n^2)**: Quadratic (e.g., bubble sort, nested loops).
- **O(2^n)**: Exponential (e.g., recursive solutions to some problems like the Tower of Hanoi, Fibonacci series).

Complexity	Example	Explanation
O(1)	Accessing an array element	Constant time (no loops)
O(log n)	Binary search	Halving input each iteration
O(n)	Linear search	Single loop
O(n log n)	Merge sort	Divide-and-conquer with merging
O(n^2)	Bubble sort	Two nested loops
O(2^n)	Generating all subsets	Recursive tree (double choices)

-



Data Structures	Algorithms	Concepts
Linked Lists	Breadth-First Search	Bit Manipulation
Trees, Tries, & Graphs	Depth-First Search	Memory (Stack vs. Heap)
Stacks & Queues	Binary Search	Recursion
Heaps	Merge Sort	Dynamic Programming
Vectors / ArrayLists	Quick Sort	Big O Time & Space
Hash Tables		

Walk through your brute force with BUD(Bottle neck,Unused, Duplicate)

optimization or try some of these ideas:

• Look for any unused info. You usually need all the information in a problem.

• Solve it manually on an example, then reverse engineer your thought process.

• How did you solve it? ⇒ Solve it"incorrectly" and then think about why the algorithm fails.

• Can you fix those issues? ⇒ Make a time vs. space tradeoff. Hash tables are especially useful!

shared_ptr, weak_ptr, unique_ptr

unique_ptr

=> Only one unique_ptr can own the object at a time.

Feature : 1. Automatically delete when goes out of scope.
2. Non-Copyble.
3. Move support

Ex. Unique_ptr<myclass> obj (new myclass)
Unique_ptr<myclass> obj = make_unique< myclass>;
Auto obj = make_unique< myclass>;

Shared_ptr

=>multiple reference count for same object.

Features: 1. Shared ownership, tracks reference count.
2. Deletes the object when the last shared_ptr is destroyed.

weak_ptr

=>std::weak_ptr: Non-owning reference to a shared_ptr-managed object, used to break circular references.

Const	constexpr:
It can initialized at compile/ run time, it also immutable, i.e. not necessary when to compute like compile Or run time	Ensures that a variable, function, or object's value/expression is **computed at compile time** and is immutable.
Performance not matter	It used for performance

what diff sql & nosql
=>

SQL	NO SQL

It has table with fix rows & column	It is Key-value pair of JSON
Schema base , Altering table is difficult	No Schema , easy to add new field on fly.
Query to access data,	User specific API, less SQL layered
DBA expert required	No DBA required, simple
Slower , choose when u have fix size	faster, choose when u have variable size.

Pattern	Category	Summary	When to Use
Singleton	Creational	Ensures a class has only one instance with global access, implemented in C++ by privatizing the constructor, using a static method (often with a thread-safe static local variable in C++11+), and managing lifetime to avoid multiples.	When exactly one instance is needed, like for shared resources such as loggers or configuration managers.
Factory Method	Creational	Defines an interface for object creation in a superclass, letting subclasses decide the concrete type; in C++, uses a pure virtual method in a base class returning a pointer/reference, overridden by subclasses.	When a class can't anticipate needed object types, e.g., in frameworks where subclasses specify instances.
Abstract Factory	Creational	Produces families of related objects without specifying concretes; in C++, an abstract base with factory methods for family members, concrete factories creating specifics, used via the abstract	When supporting product families (e.g., UI for different OSes) without hardcoding

		interface.	classes.
Builder	Creational	Constructs complex objects step-by-step for varied representations; in C++, a Builder class with incremental setters, a build() method, and optional director for guidance.	When objects have many optional parameters or multi-step construction, like documents or configs.
Adapter	Structural	Enables incompatible interfaces to work together; in C++, a wrapper class inheriting the target interface, holding the adaptee, and delegating/translating calls (via composition).	When integrating existing classes with mismatched interfaces, e.g., third-party libraries.
Decorator	Structural	Adds behaviors dynamically by wrapping objects; in C++, concrete decorators inherit from base component, wrapping others to extend functionality at runtime.	When adding responsibilities at runtime without impacting others, like UI extensions (borders, scrolls).
Facade	Structural	Simplifies complex subsystem interfaces; in C++, a single class encapsulating subsystem interactions with high-level methods hiding internals.	When providing a unified interface to complex subsystems, like API wrappers.
Proxy	Structural	Provides a placeholder for another object to control access; in C++, proxy implements subject's interface, holds reference, intercepts calls for lazy loading or security.	When controlling object access, e.g., remote proxies, on-demand loading, or security.
Observer	Behavioral	Defines a subscription for notifying observers of subject changes; in C++, subject holds observer list (e.g., std::vector<shared_ptr>), with attach/detach/notify methods calling virtual updates.	When notifying dependents of changes without coupling, like event systems or MVC.

Strategy	Behavioral	Defines interchangeable algorithm families; in C++, strategy interface with algorithm method, concrete strategies, context holding/delegating to a strategy pointer for runtime swaps.	When selecting algorithm variants dynamically, e.g., sorting or payment methods.

ByGrok:C++interviewquestionanswersforexperience

1. What are the key differences between C and C++?

Answer:

- **Paradigm**: C is procedural, while C++ supports both procedural and object-oriented programming (OOP) with features like classes, objects, and inheritance.
- **Memory Management**: C uses manual memory management (e.g., malloc/free), while C++ provides new/delete and smart pointers for safer memory management.
- **Features**: C++ supports polymorphism, encapsulation, templates, and the Standard Template Library (STL), which are absent in C.
- **Function Overloading**: C++ allows function and operator overloading, while C does not.
- **Exception Handling**: C++ has try-catch blocks for exception handling, whereas C relies on error codes.

2. Explain the concept of a virtual function and why it's used.

Answer: A virtual function in C++ is a member function in a base class that can be overridden in a derived class. It enables **runtime polymorphism**, allowing the correct function to be called based on the object's actual type, not the pointer/reference type.

Usage: Declared with the virtual keyword in the base class. When a derived class overrides it, the call is resolved via a **vtable** (virtual table) at runtime.

- **Example**:

```
class Base {
public:
    virtual void show() { std::cout << "Base\n"; }
};
class Derived : public Base {
public:
    void show() override { std::cout << "Derived\n"; }
};
int main() {
    Base* ptr = new Derived();
    ptr->show(); // Outputs "Derived" due to virtual function
    delete ptr;
    return 0;
}
```

- **Why Used**: To achieve dynamic dispatch, enabling flexible and extensible code in inheritance hierarchies.

3. What is the difference between delete and delete[]?

Answer:

- **delete**: Used to deallocate memory for a single object allocated with new. It calls the destructor of the object.

- **delete[]**: Used to deallocate memory for an array of objects allocated with new[]. It calls the destructor for each object in the array.
- **Key Difference**: Using delete on an array (new[]) or delete[] on a single object (new) leads to undefined behavior. Always match new with delete and new[] with delete[].
- **Example**:
- cpp
- CollapseWrap
- Copy

```cpp
int* single = new int;

int* array = new int[10];

delete single;   // Correct

delete[] array;  // Correct
```

4. What are smart pointers, and how do they differ from raw pointers?

Answer: Smart pointers are C++ objects that manage the lifetime of dynamically allocated memory, preventing memory leaks. They are part of the STL (<memory>).

- **Types**:
- **std::unique_ptr**: Exclusive ownership, non-copyable, movable. Deletes the object when the pointer goes out of scope.
- **std::shared_ptr**: Shared ownership, tracks reference count. Deletes the object when the last shared_ptr is destroyed.
- **std::weak_ptr**: Non-owning, reference to a shared_ptr-managed object, used to break circular references.
- **Differences from Raw Pointers**:
- **Ownership**: Smart pointers automatically manage memory; raw pointers require manual delete.
- **Safety**: Smart pointers prevent dangling pointers and memory leaks.

- **Cost**: Smart pointers have overhead (e.g., reference counting in shared_ptr).
- **Example**:
- cpp
- CollapseWrap
- Copy
- `std::unique_ptr<int> uptr = std::make_unique<int>(10); // Auto-deleted`
-
- `std::shared_ptr<int> sptr = std::make_shared<int>(20); // Shared ownership`

5. What is RAII, and how is it implemented in C++?

Answer:RAII (Resource Acquisition Is Initialization) is a C++ idiom where resource management (e.g., memory, file handles) is tied to the lifetime of objects. Resources are acquired in the constructor and released in the destructor, ensuring automatic cleanup.

- **Implementation**:
- Use classes to encapsulate resources.
- Allocate resources in the constructor.
- Release resources in the destructor.
- **Example**:
- cpp
- CollapseWrapRun
- Copy
- `class FileHandler {`
-
- `    FILE* file;`
-
- `public:`
-
- `    FileHandler(const char* name) : file(fopen(name, "r")) {}`
-
- `    ~FileHandler() { if (file) fclose(file); }`
-
- `};`
-
- `int main() {`

```
    FileHandler fh("example.txt"); // File opened in constructor, closed in destructor

    return 0; // File automatically closed when fh goes out of scope

}
```

- **Benefits**: Prevents resource leaks, simplifies error handling, and ensures exception safety.

6. Explain the difference between const and constexpr in C++.

Answer:

- **const**:
- Indicates a variable's value cannot be modified after initialization.
- Can be evaluated at runtime or compile-time, depending on context.
- Example: const int x = computeValue(); (runtime initialization).
- **constexpr**:
- Indicates a variable or function must be evaluated at compile-time.
- Used for performance optimization and compile-time computations.
- Example: constexpr int square(int n) { return n * n; } constexpr int x = square(5); (x = 25 at compile-time).
- **Key Difference**: const ensures immutability, while constexpr ensures compile-time evaluation.

7. What is the role of the volatile keyword in C++?

Answer: The volatile keyword tells the compiler that a variable's value may change unexpectedly (e.g., by hardware or another thread), preventing aggressive optimizations.

- **Usage**:
- Common in embedded systems for memory-mapped I/O.
- Prevents the compiler from caching the variable's value in registers.

- **Example**:
- cpp
- CollapseWrap
- Copy
- ```
  volatile int* statusRegister = (int*)0x1234; // Hardware register
  
  while (*statusRegister) {} // Compiler won't optimize away the read
  ```
- **Note**: volatile is not a substitute for thread synchronization (use std::atomic for that).

8. What is a vtable, and how does it work in C++?

Answer: A **vtable** (virtual table) is a mechanism used to implement runtime polymorphism for virtual functions.

- **How it Works**:
- For each class with virtual functions, the compiler creates a vtable containing pointers to the virtual functions.
- Each object of the class contains a hidden pointer (vptr) to its class's vtable.
- When a virtual function is called via a base-class pointer/reference, the vptr is used to look up the correct function in the vtable.
- **Overhead**: Adds memory (vptr per object, vtable per class) and runtime cost (indirect function calls).
- **Example**: See the virtual function example above (question 2).

9. What are templates in C++? Explain with an example.

Answer: Templates enable generic programming by allowing functions or classes to work with any data type.

- **Types**:
- **Function Templates**: Generic functions.
- **Class Templates**: Generic classes.
- **Example** (Function Template):
  ```

- cpp
- CollapseWrapRun
- Copy

```cpp
template <typename T>

T max(T a, T b) {

    return (a > b) ? a : b;

}

int main() {

    std::cout << max(5, 10) << "\n";         // int: 10

    std::cout << max(3.14, 2.71) << "\n";    // double: 3.14

    return 0;

}
```

- **Benefits**: Code reusability, type safety, and performance (compile-time resolution).
- **Drawbacks**: Complex error messages, increased compile time.

10. What is the difference between std::vector and std::array?

Answer:

- **std::vector**:
- Dynamic array, resizable at runtime.
- Allocates memory on the heap.
- Provides bounds checking in debug mode (e.g., at()).
- Example: std::vector<int> vec = {1, 2, 3}; vec.push_back(4);
- **std::array**:
- Fixed-size array, size defined at compile-time.
- Allocates memory on the stack.

- Provides bounds checking with at().
- Example: std::array<int, 3> arr = {1, 2, 3};
- **Key Difference**: vector is dynamic and flexible, while array is fixed-size and more efficient for small, fixed collections.

11. How does exception handling work in C++?

Answer: Exception handling in C++ uses try, catch, and throw to manage errors.

- **Mechanism**:
- Code that might throw an exception is placed in a try block.
- If an exception is thrown (via throw), control transfers to the matching catch block.
- Uncaught exceptions propagate up the call stack, potentially terminating the program.
- **Example**:
- cpp
- CollapseWrap
- Copy

```cpp
try {

    throw std::runtime_error("Error occurred!");

} catch (const std::runtime_error& e) {

    std::cerr << e.what() << "\n"; // Outputs: Error occurred!

}
```

- **Best Practices**:
- Use specific exception types.
- Ensure exception safety (e.g., RAII for resource management).
- Avoid throwing in destructors.

12. What is the Rule of Three/Five/Zero in C++?

Answer:

- **Rule of Three** (C++98/03): If a class defines any of the following, it should define all three:
- Destructor
- Copy constructor
- Copy assignment operator
- Reason: To manage resources (e.g., dynamic memory) correctly.
- **Rule of Five** (C++11 onwards): Adds two more for move semantics:
- Move constructor
- Move assignment operator
- Reason: To support efficient resource transfer with move semantics.
- **Rule of Zero**: Design classes to avoid manual resource management by using RAII (e.g., smart pointers, STL containers), letting the compiler generate default implementations.
- **Example** (Rule of Three):
- cpp
- CollapseWrap
- Copy
- ```
  class Resource {
  
      int* data;
  
  public:
  
      Resource() : data(new int) {}
  
      ~Resource() { delete data; } // Destructor
  
      Resource(const Resource& other) : data(new int(*other.data)) {} // Copy constructor
  
      Resource& operator=(const Resource& other) { // Copy assignment
  
          if (this != &other) {
  
              delete data;
  ```
  ```

```
            data = new int(*other.data);

        }

        return *this;

    }

};
```

13. What is the difference between static and dynamic polymorphism?

Answer:

- **Static Polymorphism** (Compile-time):
- Resolved at compile-time using templates or function overloading.
- Example: Template functions, overloaded functions.
- Benefits: Faster (no runtime overhead), inlineable.
- **Dynamic Polymorphism** (Runtime):
- Resolved at runtime using virtual functions and inheritance.
- Example: Virtual function calls via base-class pointers.
- Benefits: Flexible for runtime object types, but has vtable overhead.
- **Example**:
- cpp
- CollapseWrap
- Copy

```
// Static polymorphism (template)

template <typename T>

void print(T val) { std::cout << val << "\n"; }

// Dynamic polymorphism

class Base { virtual void show() { std::cout << "Base\n"; } };
```

```
class Derived : public Base { void show() override { std::cout << "Derived\n"; } };
```

14. Explain move semantics and std::move.

Answer: Move semantics (introduced in C++11) allow transferring resources from one object to another without copying, improving performance.

- **How it Works**:
- A move constructor/assignment operator transfers ownership of resources (e.g., pointers) instead of copying.
- std::move casts an object to an rvalue reference, enabling move semantics.
- **Example**:
- cpp
- CollapseWrapRun
- Copy

```
class MyString {

    char* data;

public:

    MyString(const char* str) : data(strdup(str)) {}

    MyString(MyString&& other) noexcept : data(other.data) { other.data = nullptr; } // Move constructor

    ~MyString() { free(data); }

};

int main() {

    MyString s1("Hello");

    MyString s2 = std::move(s1); // Move s1's resources to s2
```

```
    // s1 is now in a valid but unspecified state

    return 0;

}
```

- **Benefits**: Reduces unnecessary copying, especially for large objects.

15. What is the difference between std::mutex and std::atomic for thread safety?

Answer:

- **std::mutex**:
- Used for protecting shared resources in multi-threaded code.
- Provides mutual exclusion, ensuring only one thread accesses a critical section.
- Example: std::lock_guard<std::mutex> lock(mtx);
- Use Case: Protecting complex operations or shared data structures.
- **std::atomic**:
- Provides atomic operations (indivisible) for basic types (e.g., int, bool).
- Avoids locks, reducing contention.
- Example: std::atomic<int> counter(0); counter++;
- Use Case: Simple counters or flags in concurrent code.
- **Key Difference**: mutex is for coarse-grained locking, while atomic is for lock-free, fine-grained operations.

16. How would you optimize a C++ program?

Answer: Optimization strategies depend on the context, but common approaches include:

- **Algorithmic Improvements**: Use efficient algorithms/data structures (e.g., replace linear search with binary search).
- **Memory Management**:
- Minimize dynamic allocations using stack-based storage or std::array.

- Use smart pointers to avoid leaks.
- Reserve capacity in std::vector to avoid reallocations.
- **Code-Level Optimizations**:
- Use const and constexpr for compile-time computations.
- Avoid unnecessary copies with move semantics or pass-by-reference.
- Inline small, frequently called functions.
- **Profiling**: Use tools like gprof, Valgrind, or Intel VTune to identify bottlenecks.
- **Multithreading**: Parallelize tasks using std::thread or std::async for CPU-bound work.
- **Compiler Optimizations**: Enable -O2 or -O3 flags, use profile-guided optimization (PGO).

17. What are the differences between public, protected, and private inheritance?

Answer:

- **Public Inheritance**:
- Public members of the base class remain public; protected members remain protected.
- Models an "is-a" relationship (e.g., Dog is-a Animal).
- Example: class Dog : public Animal { ... };
- **Protected Inheritance**:
- Public and protected members of the base class become protected in the derived class.
- Used rarely, typically for implementation inheritance.
- Example: class Dog : protected Animal { ... };
- **Private Inheritance**:
- Public and protected members of the base class become private in the derived class.
- Models a "has-a" relationship (implementation detail, not exposed).
- Example: class Dog : private Animal { ... };
- **Key Difference**: Affects accessibility of base-class members in the derived class and its clients.

18. What is undefined behavior in C++? Give examples.

Answer: Undefined behavior (UB) occurs when a program's behavior is unpredictable due to violating C++ standard rules. The program may crash, produce incorrect results, or appear to work.

- **Examples**:
- Dereferencing a null or dangling pointer: int* p = nullptr; *p = 5;
- Accessing an array out of bounds: int arr[5]; arr[10] = 0;
- Using a variable after its lifetime ends: int& ref = *new int; delete &ref; ref = 5;
- Modifying a const object: const int x = 10; *(int*)&x = 20;
- **Avoiding UB**: Use modern C++ features (smart pointers, bounds-checked containers) and tools like UBSan or static analyzers.

19. Explain the STL and its main components.

Answer: The **Standard Template Library (STL)** is a collection of generic algorithms, containers, and iterators in C++.

- **Main Components**:
- **Containers**: Store data (e.g., std::vector, std::map, std::set, std::list).
- **Algorithms**: Operate on containers (e.g., std::sort, std::find, std::transform).
- **Iterators**: Provide a way to traverse containers (e.g., begin(), end()).
- **Function Objects (Functors)**: Callable objects for algorithms (e.g., std::less).
- **Example**:
- cpp
- CollapseWrap
- Copy

```cpp
std::vector<int> vec = {5, 2, 9, 1};

std::sort(vec.begin(), vec.end()); // Sort using STL algorithm

for (auto x : vec) std::cout << x << " "; // Outputs: 1 2 5 9
```

- **Benefits**: Reusability, type safety, and performance.

20. How would you implement a thread-safe singleton in C++?

Answer: A thread-safe singleton ensures a single instance of a class across threads. C++11 and later simplifies this with static initialization.

- **Implementation** (Meyers' Singleton, thread-safe in C++11+):
- cpp
- CollapseWrap
- Copy

```cpp
class Singleton {

public:

    static Singleton& getInstance() {

        static Singleton instance; // Static initialization is thread-safe

        return instance;

    }

    // Delete copy/move operations

    Singleton(const Singleton&) = delete;

    Singleton& operator=(const Singleton&) = delete;

private:

    Singleton() {} // Private constructor

};
```

- **Why Thread-Safe**: C++11 guarantees that static local variables are initialized in a thread-safe manner.

- **Alternative**: Use std::call_once or double-checked locking (with std::atomic) for older C++ versions.

Tips for C++ Interview Success

- **Demonstrate Experience**: Highlight projects where you used C++ (e.g., performance-critical systems, embedded software).
- **Know Modern C++**: Be familiar with C++11/14/17/20 features (e.g., smart pointers, lambdas, auto, ranges).
- **Practice Coding**: Solve problems on platforms like LeetCode or HackerRank using C++.
- **Understand Low-Level Details**: Be ready to discuss memory management, pointers, and vtables.
- **Ask Questions**: Clarify requirements in system design or coding questions to show problem-solving skills.

If you'd like me to expand on any question, provide more examples, or focus on specific C++ topics (e.g., multithreading, memory management, or C++20 features), let me know! I can also generate code snippets or analyze specific scenarios if you provide details.

Observer pattern , singleton pattern

AWS Services :

- services such as Lambda, API Gateway, DynamoDB, Step Functions, ECS/Fargate, and S3.
- RESTful APIs for internal and external service integration.
  microservices multi-tenant SaaS platform.

DynamoDB

1. key-value and document data models.
2. NO SQL – no relational DB
3. Serverless - don't have to manage the underlying infrastructure.

Lambda:

1. **serverless** computing service.

2. It allows you to run code without servers.

3. You write functions , then Lambda execute them as HTTP request

API Gateway:

1. It handles API requests
2.

```cpp
struct Base1 {
    virtual ~Base1() = default;
    std::string GetName() const noexcept { return "Base1"; }
};

struct Base2 {
    virtual ~Base2() = default;
    virtual std::string GetName() const noexcept = 0;
};

struct Derived : Base2, Base1 {
    std::string GetName() const noexcept override { return "Derived"; }
};

void foo(Base1* ptr) {
    std::println("{}", ptr->GetName());
    auto b2 = dynamic_cast<Base2*>(ptr);
    if (b2) {
        auto addr = (uintptr_t)(b2);
        std::println("{:x}", addr);
    }
}

void bar(Base2* ptr) {
    std::println("{}", ptr->GetName());
    auto b1 = dynamic_cast<Base1*>(ptr);
    if (b1) {
        auto addr = (uintptr_t)(b1);
        std::println("{:x}", addr);
    }
}

void baz(Derived& d) {
    std::println("{}", d.GetName());
    auto addr = (uintptr_t)(&d);
    std::println("{:x}", addr);
}

int main() {
    Derived d;
    foo(&d);
    bar(&d);
    baz(d);
}
```

```cpp
struct Base1 {
    virtual ~Base1() = default;
    std::string GetName() const noexcept { return "Base1"; }
};

struct Base2 {
    virtual ~Base2() = default;
    virtual std::string GetName() const noexcept = 0;
};

struct Derived : Base2, Base1 {
    std::string GetName() const noexcept override { return "Derived"; }
};

void foo(Base1* ptr) {
    std::println("{}", ptr->GetName());
    auto b2 = dynamic_cast<Base2*>(ptr);
    if (b2) {
        auto addr = (uintptr_t)(b2);
        std::println("{:x}", addr);
    }
}

void bar(Base2* ptr) {
    std::println("{}", ptr->GetName());
    auto b1 = dynamic_cast<Base1*>(ptr);
    if (b1) {
        auto addr = (uintptr_t)(b1);
        std::println("{:x}", addr);
    }
}

void baz(Derived& d) {
    std::println("{}", d.GetName());
    auto addr = (uintptr_t)(&d);
    std::println("{:x}", addr);
}

int main() {
    Derived d;
    foo(&d);
    bar(&d);
    baz(d);
}
```